Youth Ministry Management App

Requirements Document - Version 5

Youth Ministry Management App - Requirements Document

Version 5 - revised after fourth owner review. Changed or newly-added content in *this* revision is marked like this throughout (earlier rounds' highlighting has been cleared so this pass is easy to spot), and summarized in the Revision Log below.

Purpose of this document: This is a complete functional and technical specification for rebuilding an existing Google Apps Script / Google Sheets youth ministry management app as a modern, cloud-hosted, zero-cost web application. It is written to be pasted directly into a new Claude project as a build prompt. It documents both (a) the exact behavior of the current live app - so nothing is lost - and (b) a set of deliberate, agreed-upon improvements over that current behavior.

The current app is called **"St Arsanius Youth Ministry"** (short name "SAY Ministry"), used by a Coptic Orthodox church to track university/college-age youth ministry members, their attendance, outreach/pastoral care, volunteer ("servant") assignments, and a shared service calendar. It is being rebuilt so that (1) the same codebase/product can be reused by other ministries with different terminology (e.g. a high-school service), and (2) the underlying data store becomes a real relational database with near-real-time sync, instead of Google Sheets.

This is a **single-tenant deployment per ministry** - not a multi-tenant SaaS platform. A multi-tenant SaaS platform is one running application and one shared database serving many unrelated organizations at once, with each customer's data logically separated by a "tenant ID" column (the model behind products like Slack or Salesforce).
Single-tenant-per-ministry, what we're building here, means this ministry gets its own dedicated database and its own dedicated deployment, completely separate from any other ministry that might someday run the same codebase. The practical effect: a simpler schema (no tenant-ID plumbing threaded through every table), data that physically cannot mix with anyone else's, but if another ministry (e.g. a High School service) ever wants this product, they get their own separate Supabase project and URL - not a switch inside this one.

Revision Log

v1 -> v2 (first owner review): added QA/test environment, cohort/ladder-position group model, refined role capability split, atomic group transition with new-cohort creation and servant-review/QR-reprint prompts, real-time QR check-in/intake pipeline, average-attendance-% bug flagged, visitors un-excluded from Attendance tab, configurable cutoff time, proposed visual enhancements, fully specified data migration and cutover runbook, plus the Bible-verse/dynamic-labels/versioning/collapsible-sections requirements.

v2 -> v3 (second owner review): confirmed the Production+QA Supabase plan (at the time, believed to fit free-tier limits - see the v4 correction below), added the position-0 group's own intake-only QR code for the welcoming-party flow, clarified that Sub-Coordinators need an additional Servant role to assign members to themselves, redesigned migration as two-way sync, and confirmed several open assumptions plus the visual-enhancements acceptance (all but dark mode).

v3 -> v4 (third owner review): corrected the Supabase free-tier claim (account-wide 2-project limit, not per-organization) and revised the environment plan to one project with two schemas; reverted migration from two-way back to one-way sync with a rule to delete untraceable new-app test rows on refresh, excluding role assignments from that sweep; confirmed the Year-0-QR wording was a typo.

v4 -> v5 (fourth owner review, this revision): - **Section 10.1 / Section 3**: broadened the migration-sweep exclusion beyond just role assignments - **all admin-configured settings/configuration tables** (`universities`, `verses`,

`actions_needed_config`, `audit_config`, in addition to the already-excluded `user_roles`, `app_settings`, `groups`, `qr_codes`) are now excluded from the ongoing refresh sweep, not just seeded once at initial migration. Only genuinely operational/transactional tables (`members`, `attendance_records`, `outreach_entries`, `service_calendar_events`, and servant contact info in `profiles`) remain subject to the automatic mirror-sync on every refresh.

---

1. Technology Stack

Layer	Choice	Notes
Frontend	**Next.js (React) + Tailwind CSS**	Reproduce the existing visual design system exactly (see Section 8) using Tailwind utility classes/theme tokens
Backend / Database	**Supabase** - hosted Postgres	Relational database; replaces Google Sheets entirely
Realtime sync	**Supabase Realtime** (Postgres logical replication)	Near-instant updates across all connected users when any record changes
Auth	**Supabase Auth**, supporting **both** Google OAuth sign-in and email/password	Google sign-in is the default/primary path, matching current UX; email/password is a supported fallback
File storage	**Supabase Storage**	Member/servant photos, calendar attachments, QR code images, app logo - replacing the current Google Drive folder + filename-matching approach
Hosting	**Vercel**, free tier	Auto-deploys Next.js; free `*.vercel.app` subdomain to start (custom domain can be added later, ~$12-15/yr, decide later)
Row-level security	**Postgres RLS policies**	Enforces the role/group permission model (Section 4) at the database layer - see Section 4.4

All of the above operate comfortably within free tiers at this app's transaction volume and data size (see Section 9.3 for the storage-headroom analysis that confirms this).

1.1 Environments: Production and QA (corrected - single shared Supabase project, schema-separated)

Correction from the previous revision: Supabase's free-project limit (2 active projects) is **counted across your whole account** - every organization where you're an Owner/Admin shares the same 2-project cap, not 2-per-organization as previously stated here. Since you already have one active project elsewhere on your account, only **one** more free project

slot is available in total. Two brand-new separate projects for this app's Production and QA (as originally proposed) would bring your account to 3 active projects, exceeding the free limit and requiring a paid plan for at least one of them. Apologies for the earlier inaccurate claim - corrected here with a source: Supabase Billing FAQ.

Revised plan - one Supabase project, two schemas - fits entirely free: - **A single new Supabase project** for this app (your account's one remaining free slot), containing **two separate Postgres schemas**: `prod` and `qa`, each with its own full copy of every table (`groups`, `members`, `attendance_records`, etc.) - completely separate data, completely independent from one another, just sharing the same underlying project/database instance. - **Storage**: separate Supabase Storage buckets per environment (e.g. `prod-photos` / `qa-photos`) within that one project - fully supported, no limit issue. - **Auth**: Supabase Auth's user accounts are shared across the whole project (not schema-scoped) - which is actually convenient here: you and your testers sign in with the same account in both environments, and each environment's own `user_roles` table (living in its own schema) independently controls what that same person can do in `prod` vs. `qa`. - **Still two separate Vercel deployments** (free, no limit concern there) - a production site and a QA site, each configured (via environment variables) to talk to its matching schema. - New features/schema changes are built and verified against `qa` first, then the same migration is re-applied against `prod` once confirmed - same promotion workflow as originally planned, just routed through schemas instead of separate projects.

Trade-off, stated plainly: this gives genuine data separation (a QA mistake cannot touch `prod` rows - different tables entirely) but not full infrastructure separation - both environments share the same database compute and the same pool of login accounts. At this app's transaction volume, that's not a practical concern. If your other, unrelated Supabase project is ever retired or you're willing to pay for a second organization's Pro plan later, moving from schema-separation to fully separate projects is a straightforward one-time export/import - not a dead end, just not necessary today to stay at zero cost.

---

2. Branding & Terminology Configuration

Everything about the app's identity and vocabulary must be configurable per deployment, stored in a single-row (or key/value) **app settings** table - not hardcoded.

2.1 Configurable fields

Setting	Generic default	This deployment's value (St Arsanius / SAY)
App title (long)	"Service Members Ministry"	"St Arsanius Youth Ministry"
Short name (PWA / home-screen name)	"Members Ministry"	"SAY Ministry"
Subtitle (shown under the header title)	"Servant Dashboard"	"Youth Servant Dashboard"
Logo / icon image ("Service Patron Saint")	placeholder	St. Arsanius icon/logo image (currently `St Arsenius Icon.png` / `St Arsenius Logo.png`)
Theme color	`#1e3a5f` (deep navy)	`#1e3a5f` (keep as-is)

"Group" label	"Group"	"Cohort" (changed from "Year")
"Member" label	"Member"	"Youth"
App version (independent counter)	"1.0"	starts at **"4.0"** - see Section 13 for the versioning workflow
Same-service-day attendance cutoff time	21:00 (9 PM)	21:00 (9 PM), America/New_York

- The **logo/icon storage mechanism** (an uploaded image referenced by the app, displayed 60x60px circular in the header, plus home-screen/PWA icon variants) stays conceptually the same as today - just stored in Supabase Storage instead of a hardcoded Google Drive thumbnail URL.
- The **"Version X.X" label** in the header's top-right corner stays exactly as-is: a small static build/release version marker, not user-configurable content-wise, but its *value* is driven by the app-version workflow in Section 13, independent of Vercel's own deployment versioning.
- All screen labels that currently say "Year" / "Youth" should pull from these two config strings instead of being hardcoded, so a different deployment (e.g., a high-school service) can relabel to "Grade" / "Student" etc. without code changes. This includes action labels - e.g., the "Load Youth Data" button must render as "Load [Member label] Data" (so "Load Youth Data" here, "Load Member Data" generically) - anywhere the Member/Group label appears in UI copy, not just in data displays.
- **Verses list** (new): a simple admin-managed list of Bible verses (or equivalent inspirational text for a non-Christian deployment - keep the concept generic), stored in a `verses` table (Section 3.13). See Section 6.1 for how it's used.

2.2 Groups, Cohorts, and the Ladder Position Model (revised)

Unlike the current app (which has 5 hardcoded, structurally-duplicated Google Sheets - one per year), **groups are rows in a database table**, not separate schemas. Each group represents a specific cohort of members, identified by an (optional but recommended) **cohort birth year**, plus a **ladder position** describing where that cohort currently sits in the progression. A group's *display name* can be generated automatically from a configurable template combining the cohort year and position - this is exactly how this deployment's "2007 Cohort - Yr 1 -> 2007 Cohort - Yr 2" relabeling works: the cohort (the same real people) doesn't change identity when it transitions, only its ladder position advances, and its name is regenerated to match.

This deployment's current ladder (positions, not fixed group names - names shift every transition):

Ladder position	Cohort year	Current display name	Accessible to
0 (pre-entry)	2008	"2008 Cohort - Yr 0"	**Admin only** - hidden from everyone else, including General Coordinators
1	2007	"2007 Cohort - Yr 1"	Normal group access rules (Section 4)
2	2006	"2006 Cohort - Yr 2"	Normal group access rules

3	2005	"2005 Cohort - Yr 3"	Normal group access rules
4	2004	"2004 Cohort - Yr 4"	Normal group access rules
5+ (terminal)	2003 and every earlier cohort	"2003 Cohort and earlier - Yr 5+"	Normal group access rules

How this works mechanically: - Each cohort is its own permanent group row, keyed by `cohort_year`. It never merges with another cohort's row - even once "graduated," a cohort (e.g., born 2003) keeps its own identity for precise record-keeping and archiving. - `ladder_position` is what the transition trigger advances (0->1->2->3->4->5). Once a cohort reaches position 5 ("terminal"), further transitions no longer move it - it just sits at position 5 indefinitely (archived later via Section 3.3.1, per-cohort or in batch). - Because multiple cohorts eventually pile up at position 5 over the years, the **terminal display is an aggregate view**: everyday screens show all terminal-position cohorts collapsed under one umbrella label - "**[oldest un-archived cohort year] Cohort and earlier - Yr 5+**" - computed live, not stored, so it never goes stale. Admin screens (for archiving) can still drill into each individual cohort year separately. - **Ladder position 0 is special**: it's a holding pen for members who haven't started at the group's normal entry point yet (in this deployment: high-schoolers not yet in university). It **does** get its own public, no-login QR code (Section 6.11, Section 6.15) - used at the ministry's annual welcoming party (July/August) to invite incoming members to submit their complete record information ahead of officially joining. That QR opens **intake only**: no "find your name and check in" option, since these individuals aren't tracked for attendance at this stage. Submissions go straight into this holding-pen group. Regardless of how a record gets in, this group is invisible to every other role - it doesn't appear in the landing-page group selector, no Sub-Coordinator/Servant/General-Coordinator role can be scoped to it, and only Admins can browse its roster. - **Group naming**: `app_settings.group_name_template` holds a template string, e.g. `"{cohort_year} Cohort - Yr {position_label}"`, applied automatically whenever the transition trigger advances a cohort's position. A deployment that doesn't want cohort-year-based naming can simply leave `cohort_year` null on its groups and skip the template - names then stay exactly as manually set (e.g., a plain "Grade 9" style deployment). - **New Yr-0 creation**: when the transition trigger runs and the current position-0 cohort advances to position 1, the process prompts the Admin to create the next incoming position-0 cohort (defaulting the suggested cohort year to "previous Yr 0's cohort year + 1," editable) - see Section 5.

---

3. Data Model

All entities below are Postgres tables. Column lists describe intent/type, not final SQL syntax - see the companion **DATABASE_SCHEMA.md** for the actual implementable DDL.

3.1 `app_settings`

Single-row table (or key/value table) holding everything in Section 2.1, plus `app_version` (text, e.g. "4.0"), `group_name_template` (text), `same_day_cutoff_time` (time, default 21:00), `timezone` (text, default "America/New_York"), and the "current service year" label used to generate QR code names dynamically (replacing the hardcoded `"25-26"` string in the current app).

3.2 `groups` (revised)

- `id`
- `cohort_year` (int, nullable) - the birth year (or other stable cohort identifier) this group represents

- `ladder_position` (smallint) - 0 = pre-entry/hidden, 1..N-1 = progressing, N = terminal
- `name` (text) - the display name; auto-regenerated from the template at transition time when `cohort_year` is set, otherwise manually managed
- `is_archived` (bool, default false) - true once this cohort has been fully archived out (Section 3.3.1)

3.3 `members` (replaces "Youth" / per-year "Master List")

One table for all members across all groups - no more one-sheet-per-year duplication. - `id` - `group_id` (FK -> groups) - `photo_url` (FK-style reference into Supabase Storage - **not** filename-matching like today) - `full_name` - `phone` - `email` - `university_college_id` (FK -> `universities`) - *note: keep this field's real-world label configurable too, since a non-university service wouldn't call it "University/College"* - `program_of_study` - `date_of_birth` - `father_of_confession` - `home_address` - `is_visitor` (bool) - a visitor stays fully visible to servants everywhere in the app. Their attendance **is** now tracked and shown, tagged as "Visitor" (Section 6.5, Section 7.3) - this is a change from the original spec, which incorrectly excluded them from the Attendance tab entirely. They remain excluded from Dashboard aggregate stat counts and from the Actions Needed algorithm, matching today's behavior - flagged as an assumption in Section 12, confirm if this should change too. - `gender` - `comments` (member/registration comments, read-only after creation, matching current behavior) - `assigned_servant_id` (FK -> users, nullable) - `servant_comments` - `is_new_assignment` (bool) - notification flag, cleared when the assigned servant views/dismisses it - `status` - `active` | `archived` (see Section 3.3.1) - `created_at`, `updated_at`

3.3.1 Archive vs. permanent delete: - **Archive**: available for members in a terminal-position group, individually or in batch, triggered explicitly by a Coordinator/Admin action (distinct from the yearly transition trigger). Archiving sets `status = 'archived'` - the member is removed from active rosters, dashboards, and stat counts, but their full record **and all historical attendance/outreach data are preserved** in the database, recoverable later. Confirmed: "if we ever run out of database space, we can delete them then" - archived rows are the long-term home for graduated members; permanent deletion is a separate, rare, manual DB-admin action outside the app's normal UI, not a feature to build. - **Permanent delete**: a separate, **Admin-only** "delete member record" action, usable on a member in **any** group (not just a terminal one) - intended for correcting mistakes: a record entered in error, or a test record. This performs an actual irreversible delete and must require an explicit confirmation step. Confirmed: normal day-to-day removal from an active group does *not* happen - that's what the `is_visitor` flag is for (kept visible, excluded from stats). This permanent-delete tool exists specifically as a correction mechanism, not a routine "member left" workflow.

3.4 `universities`

- `id`, `name`, `proximity` (`Local` | `Regional` | `Abroad`)

3.5 `users` (Supabase Auth users - Admins, Coordinators, Servants; NOT members)

- `id` (Supabase Auth UID), `full_name`, `phone`, `email`, `father_of_confession`, `gender`, `photo_url`

3.6 `user_roles` - the permission model (see Section 4)

- `id`, `user_id` (FK -> users), `role` (`admin` | `general_coordinator` | `sub_coordinator` | `servant`), `group_id` (FK -> groups, **required** for `sub_coordinator` and `servant`, **null** for `admin`/`general_coordinator`)

A user can hold multiple rows in this table simultaneously - e.g., Servant@2007-Cohort + Sub-Coordinator@2007-Cohort, or Servant@Group1 + Admin. A user can also hold the **same role type scoped to**

more than one group - this is exactly how "exception access" (e.g., a Year 1 servant who also needs visibility into Year 3) is handled: grant them an additional role row (typically Sub-Coordinator, so they get full data access without being pulled into that group's servant-assignment pool) scoped to the second group. No separate mechanism is needed. Effective permissions/access are the union of everything their role rows grant (see Section 4).

3.7 `attendance_records`

- `id`, `member_id` (FK, nullable) OR `user_id` (FK, nullable) - one or the other, representing member attendance vs. servant attendance - `service_date` (date). A row's existence = present on that date. Same "presence-row" model as today.

3.8 `outreach_entries`

- `id`, `member_id` (FK), `servant_id` (FK -> users), `date_time`, `type` (free text), `notes`, `follow_up_due` (date, nullable), `follow_up_dismissed_at` (timestamp, nullable)
- Row-ownership rule carried forward: only the creating servant can edit/delete their own outreach entries.

3.9 `service_calendar_events`

- `id`, `title`, `description`, `type` (enum: `Trip` | `Outing` | `Group Discussion` | `Speaker Session` | `Event` | `Holiday`), `start_date`, `end_date`, `start_time` (nullable), `end_time` (nullable), `all_day` (bool), `location`, `attachment_url` (nullable), `created_by` (FK -> users), `created_at`

3.10 `actions_needed_config`

Per-proximity thresholds, admin-editable via a real UI (Section 6.9). Columns: `proximity` (`Local`|`Regional`|`Abroad`|`Unknown`), `min_presence_count`, `min_absence_weeks`, `min_outreach_weeks`.

Seed this table with the current live production values:

```
Local: min_presence_count=0, min_absence_weeks=3, min_outreach_weeks=4 Regional:
min_presence_count=0, min_absence_weeks=3, min_outreach_weeks=4 Abroad: min_presence_count=0,
min_absence_weeks=6, min_outreach_weeks=4 Unknown: min_presence_count=0, min_absence_weeks=3,
min_outreach_weeks=4
```

The Dashboard's "?" help modal for Actions Needed must read these same values live at render time - the explanatory text is generated from this table, not hardcoded, so an admin editing thresholds can never leave the help text out of sync again.

Seeded once at initial migration, then admin-owned going forward - excluded from the ongoing refresh sweep (Section 10.1), same as `universities`, `verses`, and `audit_config`.

3.11 `audit_log`

- `id`, `timestamp`, `user_id`, `action_type`, `group_id` (nullable), `details` (jsonb)
- Action types: `APP_ACCESS, GROUP_SELECTED, MEMBER_EDITED, SERVANT_ASSIGNED, OUTREACH_ADDED, OUTREACH_UPDATED, OUTREACH_DELETED, MEMBER_PHOTO_UPLOADED, SERVANT_PROFILES_VIEWED, SERVANT_ATTENDANCE_VIEWED, SERVANT_EDITED, SERVANT_GROUP_UPDATED, SERVANT_PHOTO_UPLOADED, SERVANT_DELETED, ADMIN_ACCESS_MAINTENANCE, ADMIN_UNIVERSITIES_MAINTENANCE, ATTENDANCE_ADDED,`

`ATTENDANCE_REMOVED, CALENDAR_EVENT_CREATED, CALENDAR_EVENT_UPDATED, CALENDAR_EVENT_DELETED, MEMBER_ARCHIVED, MEMBER_DELETED, GROUP_TRANSITION_RUN`

- Each action type individually toggle-able via an `audit_config` table (`action_type`, `enabled`, `description`), admin-editable.

3.12 `qr_codes` (revised)

- `id`, `group_id` (FK, nullable - null for the "Servants" QR), `label` (generated from group/servants name + current service-year setting), `image_url`, `check_in_url` (the stable public URL/token this code points to - see Section 6.11), `printed_at` (timestamp, nullable - tracks whether this code's current label has been printed, so the post-transition reprint prompt (Section 5) can tell you exactly which codes are stale), `flow_type` (`check_in_and_intake` | `intake_only` - the position-0 group's QR is `intake_only`, since attendance isn't tracked for that group; every other group's QR is `check_in_and_intake`)

3.13 `verses` (new)

- `id`, `text`, `reference` (e.g. "John 3:16"), `is_active` (bool) - admin-managed list; a random active verse is selected each time a user clicks "Load [Member] Data" on the landing page (Section 6.1).

---

4. Roles & Permissions Model (refined)

This is a **deliberate redesign**, replacing the current app's confusing two-layer system (an app-level "Permissions" sheet allowlist *plus* independent native Google-Sheet sharing, with separate and easily-desynced "Serving Year" vs. "Access Year" concepts). The new model is simpler and enforced consistently.

4.1 Roles and their capabilities (capability split added)

- **Admin** - full access to everything, including admin-only screens (Access Maintenance, Universities Maintenance, Calendar Maintenance, Actions Needed Config, Audit Logs, Audit Report, Group Transition tool, Verses Maintenance).
- **General Coordinator** - full read/write access to **all groups'** member/attendance/outreach data, sees the Coordinator Corner, but not admin-only screens (unless also holding the Admin role). **Only General Coordinators (and Admins) can reassign a servant's serving group or remove a servant** via Servant Profiles & Assignments (Section 6.13).
- **Sub-Coordinator** - same data access as General Coordinator, but scoped to **exactly one group**. Can assign/unassign **members** to servants within their group (from the Dashboard's unassigned-members list or a member's detail view) - this is a normal part of managing group data. Sub-Coordinators **cannot** reassign a servant's own group or remove a servant - that stays General-Coordinator/Admin-only. **Assigning members to themselves**: since the assignment dropdown's semantics (caseload counts, gender filtering, "who's an active servant here") are inherently tied to holding the Servant role, a Sub-Coordinator who wants to personally take on members needs to also hold a Servant role row scoped to that same group - granted once via Access Maintenance, same as any other role grant (Section 4.2). This is expected to be a common, unremarkable combination.
- **Servant** - the base role. Scoped to one group (their "serving group"), which determines: (a) which group's data they can view/edit, and (b) that they appear in that group's servant-assignment dropdowns.

4.2 Multiple roles per user

A user can hold any combination of role rows, including the **same role type at more than one group** (Section 3.6) - this is how cross-group access exceptions (e.g., "a Year 1 servant who also needs Year 3 visibility") are handled: grant an additional role row scoped to the second group, through the same Access Maintenance screen used for everything else. Effective access = the union of every role's group scope; effective capabilities = the union of every role's permissions.

4.3 Group transition and role scope

When the group-transition trigger fires (Section 5), **Sub-Coordinators and Servants scoped to a group move with their cohort** - their `group_id` follows the same underlying group row as it advances ladder position, exactly like a Servant's member assignments do.

4.4 Enforcement

Enforce this entirely through **Postgres Row-Level Security policies**, keyed off `auth.uid()` joined against `user_roles`. This collapses the current app's fragile two-layer model into a single, consistently-enforced system - write the policy once per table, and it protects every query path, including future features.

---

5. Group Transition (expanded)

- **Trigger**: a single explicit admin action ("Transition Groups" or similar), not automatic, not scheduled.
- **Effect of one click**: every progressing group advances one ladder position at once (position 0->1->2->3->4->5), for members, their assigned servants, and Sub-Coordinators scoped to a group, per Section 4.3. Group display names regenerate from the naming template (Section 2.2) as positions change. A terminal-position (5) group is **not** advanced further - it accumulates until explicitly archived.
- **Atomicity / rollback**: the entire cascade (every group's position advance, every affected role's group re-scoping, name regeneration) runs inside a single database transaction. If any step fails partway through, Postgres rolls back the whole operation automatically - the database is left exactly as it was before the trigger ran, with a clear error shown and nothing half-applied. This is a hard requirement, not an optimization.
- **New position-0 cohort**: as part of the same transition flow, the Admin is prompted to create the next incoming pre-entry cohort - a new `groups` row at ladder position 0, with a suggested `cohort_year` defaulting to "previous position-0 cohort's year + 1" (editable before confirming). This ensures there's always a place to keep entering next year's incoming pre-university members immediately after transition day.
- **Optional post-transition servant review**: immediately after the transition completes, present an optional, skippable "Review Servant Assignments" screen - every servant listed with a "keep current group / reassign" control, defaulting to no change. This reuses the existing single-servant reassignment function under the hood; it's just surfaced as a guided batch-review step right when it's most relevant, rather than requiring the Coordinator to revisit each servant individually later via Servant Profiles & Assignments (still available for anyone skipped or handled later).
- **Post-transition QR reminder**: since every group's display name changes with its ladder position, the transition flow ends with a prompt: "Group labels have changed - reprint QR codes for: [list of affected groups]," linking directly to the Print QR Codes view (Section 6.15).
- Log a `GROUP_TRANSITION_RUN` audit event.

---

6. Feature Specifications (screen by screen)

Faithfully reproduce all of the following from the current app, using "Group"/"Member" terminology wired to the configurable labels (Section 2), unless a change is explicitly called out.

6.1 Landing / role-gated sections (expanded)

- A landing page requiring **manual group selection** before entering the main app (no auto-select, even if a user only has one group - this is intentional in the current app, carry it forward).
- **Servant Corner**: group selector + "Load [Member label] Data" button (label follows the configured Member term, e.g. "Load Youth Data" for this deployment), "Servants Directory", "Service Calendar", "QR Codes" buttons - visible to everyone.
- **Bible verse on load**: clicking the "Load [Member] Data" button selects and displays a random active verse from the `verses` table (Section 3.13) while the group's data loads - reproducing the current app's exact landing-page behavior. An admin-managed screen (simple add/edit/remove list, same pattern as Universities Maintenance) replaces direct spreadsheet editing of the Verses tab.
- **Coordinator Corner**: "Servant Profiles & Assignments" and "Servants Attendance" buttons - visible to General/Sub-Coordinators and Admins.
- **Admin Corner**: Access Maintenance, Universities Maintenance, Calendar Maintenance, Actions Needed Config, Audit Logs, Audit Report, Group Transition, Verses Maintenance - visible to Admins only. The position-0 ("Yr 0" equivalent) pre-entry group and its member-intake path are reachable only from here.
- **Back to landing page (new)**: the current app has no way to return to the landing page short of a full browser refresh (which restarts the app). The rebuild must include a persistent, always-visible control in the main app's header (e.g., clicking the logo/app title, or a dedicated "Home" icon) that returns to the landing page instantly via client-side navigation - no reload, no lost session.

6.2 Main navigation (once a group is loaded)

Five tabs: **Dashboard, Member List, Attendance, Outreach, Analytics.** Every tab includes a **"My Assigned List"** filter checkbox.

"My Assigned List" behavior: persists for the duration of the browser session (`sessionStorage`), staying ON across tab switches and data reloads within that session, resetting to OFF only when a new session starts. When active, it filters every list/stat/table app-wide to only the current user's assigned members, with the header's navy-to-pink gradient swap as the visual cue.

6.3 Dashboard tab (collapsible sections added)

- Stat cards: Total Members, Never Attended This [Service Year], Present Last [Service Day], Absent Last [Service Day].
- Proximity breakdown card: Local / Regional / Abroad / Unknown counts, with a disclaimer noting how many visitors are excluded from these counts.
- **Current Birthdays**: members with birthdays from 7 days ago through 14 days ahead (wrapping the year boundary), each with a clickable name and an "Outreach" quick-action link.
- **New Registrations (Unassigned)**: any member with no assigned servant - inline, gender-filtered servant-assignment dropdown with caseload counts. New members appear here **immediately** upon intake-form

submission (Section 6.11).

- **Actions Needed** section (Section 7.1) - color-coded, dismissible cards.
- **Collapsible sections (new)**: "New Registrations (Unassigned)," "Current Birthdays," and "Actions Needed" (and any other Dashboard section added later) each get a header-level "-"/"+" toggle to collapse/expand that section's body - so the page doesn't require excessive scrolling to reach later sections. Collapse state persists per session (same mechanism as "My Assigned List"), resetting each new session (confirmed).

6.4 Member List tab

- Search by name; collapsible advanced filter panel (Assigned Servant multi-select, University/Affiliation multi-select, Exclude Visitors, Has Photo, Gender, Proximity checkboxes), with a live active-filter-count badge.
- Responsive card grid; each card shows photo/initials avatar, name, university/program, phone (tel: link), and average attendance %. Click -> member detail.
- **Member detail view**: photo (click to upload), Full Name (always read-only), Phone, Email, University/Affiliation, Program, DOB, Father of Confession, Home Address, Registration Comments (always read-only), Assigned Servant (gender-filtered dropdown with caseload counts), Servant Comments. Edit mode highlights editable fields with the pale-yellow/amber styling described in Section 8. Buttons: Edit -> Save/Cancel/Close, plus always-visible "Prev. Outreach" / "New Outreach" shortcuts.
- Admin-only: **permanent delete** action for a mistaken/test record (Section 3.3.1), with confirmation.
- **Average attendance % - suspected bug (flagged)**: the current app's formula is `round(present dates / total tracked dates in the last 12 months) x 100`. A likely defect: the denominator counts tracked dates *before the member even joined*, unfairly lowering their percentage. The rebuild's correct rule: **the denominator only counts tracked dates from the member's registration date forward**, never earlier. Before this is considered done, spot-check a handful of real members' numbers by hand against the new calculation to confirm correctness - this applies equally to servants (Section 6.13).

6.5 Attendance tab (visitor handling reversed)

- A date-picker showing only service dates that actually have recorded attendance.
- **9 PM cutoff rule** (now configurable - see Section 7.2): preserve the same-day gating behavior, using the deployment's configured cutoff time and timezone.
- Sortable table: Name / Proximity badge / Attendance status. Three states: **Present**, **Absent**, **No-Show**, each with distinct visual treatment; clicking any status opens a confirmation dialog before writing.
- **Visitors are included in this table** (reversed from the original spec, which incorrectly excluded them entirely) - their attendance is tracked exactly like any other member's, and each visitor's row carries a visible "Visitor" tag/badge so servants can tell at a glance. Dashboard aggregate stats and the Actions Needed algorithm continue to exclude visitors, matching today's behavior (Section 3.3, Section 7.1, Section 7.3) (confirmed).

6.6 Outreach tab

- Search + filter (Member, Servant, Date range). "+ Add Outreach Entry" opens a form: Member (dropdown, defaults to the current user's assigned members, with "show all"/"show unassigned" toggles), Servant (defaults to current user), Date & Time (prefilled to now), Quick Call/Text action links, Type (free text), Notes, optional Follow-Up Due date.

- List of entries; only the creator sees Edit/Delete controls on their own entries.
- Follow-up reminders and new-assignment notifications surface on the Dashboard's Actions Needed section, scoped to entries/assignments relevant to the current user.

6.7 Analytics tab

- Data-completeness stat cards: % assigned to servants, % with phone, % with email, % with DOB, % with Father of Confession, % with photo.
- **Servant Assignments table**: every servant (grouped female-then-male, then alphabetically) with their current assigned-member count, plus an unassigned-members total row.
- Average attendance by month.

6.8 Service Calendar

- Fullscreen modal, four views: **Month, Week, List, and a "Fridays"-style filtered view** (renamed to match the deployment's actual service day).
- Event types (fixed enum): **Trip**, **Outing**, **Group Discussion**, **Speaker Session**, **Event**, **Holiday**, each with its own color pairing (unchanged from current app).
- Event form: Type*, Title*, Description (auto-template suggestions for Speaker Session / Group Discussion), Start/End Date*, All-Day toggle (default on), Start/End Time (hidden if all-day), Location, single file attachment.
- **Event creation/editing/deletion is open to all Servants** (confirmed intentional, not restricted).
- **Calendar Maintenance** (Admin Corner): bulk-preload holidays/feast days for a selected year, computing Coptic/Orthodox feast dates **algorithmically** (extending beyond the current app's hardcoded 2035 cutoff).

6.9 Actions Needed Config (help modal now live-linked)

Admin screen to view/edit the four proximity categories' thresholds (Section 3.10). **The Dashboard's "?" help modal explaining Actions Needed must be generated from these same live values**, not separately hardcoded text - editing a threshold here immediately updates what users see explained on the Dashboard.

6.10 Group Transition tool

Admin-only. A confirmation-gated action that cascades every group forward one ladder position, atomically, with the new-cohort-creation, optional servant-review, and QR-reprint-prompt steps described in Section 5.

6.11 New Member intake form (revised - see redesigned pipeline below)

Full redesigned process, replacing the current QR-scan -> Google Form -> overnight batch-copy pipeline:

Each group (and a separate one for Servants) has a QR code linking to a **public, no-login check-in page** scoped to that group - reachable at a stable, unguessable URL per group (`check_in_url` in `qr_codes`, Section 3.12). This is what gets printed and posted at the meeting, replacing the current Google Form link.

Existing member flow: scan the group's QR code -> the check-in page shows a searchable list of that group's current active members -> tap/select your name -> submit -> an `attendance_records` row is written **immediately, in real time**. No delay, no batch job - visible to servants instantly, same as walking attendance is recorded today via the Google Form, just without the separate spreadsheet hop.

New member flow: on the same check-in page, a "Don't see your name?" link leads straight into the New Member intake form - the same fields as the member schema (Section 3.3): Full Name, Phone, Email, University/Affiliation, Program, DOB, Father of Confession, Home Address, Gender, Comments. On submit: **the member record is created in the live roster immediately, AND today's attendance record is written for them, in the same operation** - both happen automatically and instantly. This fully replaces the current app's nightly Apps Script trigger (which copies "New Member Responses" rows into the Master List and back-fills their first attendance record hours later) with a real-time equivalent - new members and their first attendance show up for servants the moment they submit, not the next morning.

Reasonable field-level validation (valid phone/email formats, DOB in the past) is added at this entry point - a genuine data-quality improvement over the current disconnected Google Form.

Answering the process questions directly: QR codes are printed via the Print QR Codes view (Section 6.15); scanning one opens the check-in page above (replacing the Google Form); and yes - a new member's information and their same-day attendance are both captured automatically and immediately, with no manual or overnight step involved.

The position-0 pre-entry group (Section 2.2) gets a variant of this same flow, not a manual-only one: it has its own public QR code, but scanning it opens **intake only** - there's no "find your name and check in" option, since these individuals aren't tracked for attendance yet. This is the QR used at the ministry's annual welcoming party (July/August) to capture incoming members' complete information ahead of them officially joining at the next Group Transition. Submissions land directly in the position-0 holding pen, visible only to Admins, with no attendance record written. (Admins can also add records here manually from the Admin Corner if needed, using the same form.)

6.12 Attendance self-check-in (folded into 6.11's redesign above)

The check-in page described in Section 6.11 *is* the replacement for the standalone attendance Google Form - one QR code, one page, per group, handling both "I'm already a member, mark me present" and "I'm new here" in a single flow.

6.13 Servant Directory / Profiles & Assignments (permission split added)

- **Servant Profiles & Assignments** (Coordinator Corner): lists all servants; click any servant to view their profile. **Changing a servant's group assignment, or removing a servant entirely, is restricted to General Coordinators and Admins** - Sub-Coordinators can view this screen but cannot make these changes. (The earlier-flagged "Coming soon!" stub found in the code review is unrelated dead code with no UI entry point and is not being reproduced.)
- **Assigning members to servants** (distinct function): available to Sub-Coordinators as well as General Coordinators/Admins, scoped to their own group - triggered from the Dashboard's "New Registrations (Unassigned)" list or a member's detail view, exactly as already specified in Section 6.3/Section 6.4. This is a normal part of managing a group's data, not a servant-roster-management action.
- Two view modes: **Categorical** (grouped by serving group, then General Coordinators, then Unassigned; female servants listed before male within each group) and **Alphabetical**. Both retained.
- **Servants Directory** (read-only, from the landing page): searchable, grouped by group, with phone (tel: link) and average attendance %. Subject to the same average-attendance-% fix described in Section 6.4.
- **Servants Attendance**: same Present/Absent/No-Show pattern as member attendance, applied to servants.
- Servant detail: photo (click to upload), Full Name (read-only), Phone, Father of Confession, Gender - editable; Email is **not** editable. General-Coordinator/Admin can Remove a servant (with confirmation).

6.14 Admin screens

- **Access Maintenance**: manage who holds which role(s), scoped to which group(s), over `user_roles` (Section 4). **Example of an "exception" grant**: a servant serving the 2007 Cohort who also needs visibility into the 2005 Cohort simply gets a second role row here - e.g., an additional Sub-Coordinator grant scoped to the 2005 Cohort - through this same screen, no separate mechanism required.
- **Universities Maintenance**: CRUD grid for the affiliations list.
- **Verses Maintenance** (new): simple add/edit/remove list for the Bible verses shown on data-load (Section 6.1, Section 3.13).
- **Audit Logs** / **Audit Report**: filterable log viewer plus per-action-type enable/disable config.

6.15 QR Codes (revised)

7 QR codes in this deployment (one per group including the newly-visible-once-transitioned position-0 "2008 Cohort - Yr 0," plus one for Servants) - up from the current app's 6, since the pre-entry group didn't previously have one. Labels are generated dynamically from each group's current display name (Section 2.2) plus the configured service-year setting, replacing the current app's hardcoded "25-26" strings.

Printing (new): a "Print QR Codes" view lays out all current QR images with their labels in a print-optimized layout (browser print dialog, which also supports "save as PDF" natively - no extra infrastructure needed).

Reprint tracking (new): each `qr_codes` row tracks whether its *current* label has been printed (`printed_at`, Section 3.12). Whenever the Group Transition process changes a group's display name, its QR code is flagged as needing a reprint, and the post-transition prompt (Section 5) lists exactly which codes are now stale.

---

7. Business Rules (exact logic to preserve)

7.1 "Actions Needed" algorithm

For each active (non-archived, non-visitor) member, using the trailing 12 months of data: 1. Determine the member's `proximity` from their university/affiliation lookup (default `Unknown` if no match), and pull that proximity's thresholds from `actions_needed_config`. 2. `presence_count` = count of attendance rows in the last 12 months. 3. `current_consecutive_absences` = walking backward from the most recent service date that has *any* attendance data recorded, count consecutive dates this member was absent, stopping at their most recent Present date. 4. `outreach_is_stale` = true if the member has zero outreach entries ever, or their most recent outreach `date_time` is older than `min_outreach_weeks x 7` days. 5. **Flag the member iff all three hold**: `presence_count >= min_presence_count AND current_consecutive_absences >= min_absence_weeks AND outreach_is_stale`. 6. Flagged members are grouped by assigned servant and merged with open outreach follow-up reminders and new-assignment notifications into the same per-servant card list on the Dashboard.

Visitors are excluded from this algorithm (confirmed).

7.2 Attendance / "service date" logic (cutoff now configurable)

- A date only "counts" as a real tracked service date if at least one attendance row exists for it anywhere in the group.
- **The same-day cutoff time (default 9 PM) and the timezone it's evaluated in are both configurable per deployment** (`app_settings.same_day_cutoff_time`, `app_settings.timezone`) - different ministries may run their weekly service on a different day and want the "don't show today until service is basically over" cutoff at a different

hour.

- Average attendance % for a member/servant = `present dates / total tracked dates since that person's registration date` (see the bug-fix note in Section 6.4 - the denominator must not include dates before the person joined).

7.3 Visitor handling (reversed)

`is_visitor = true` members/rows: fully visible everywhere in the UI, **including the Attendance tab, where their attendance is now tracked and their row is tagged "Visitor"** (reversed from the original spec). They remain excluded from Dashboard aggregate stat counts and from the Actions Needed algorithm (confirmed).

---

8. Visual Design System

Reproduce the existing system exactly as a baseline - same navy identity, same layout, same component patterns - so nothing needs to be relearned.

Palette: Primary `#1e3a5f` (deep navy) -> `#2d5a7b` gradient. Filtered-state header swaps to `#c2185b` -> `#d81b60` when "My Assigned List" is active. Body background `#f5f5f5`, text `#333`.

Typography: `'Segoe UI', Tahoma, Geneva, Verdana, sans-serif` throughout.

Layout: `max-width: 1200px` centered container. Responsive card grids.

Cards, Badges, Buttons, Forms, Modals, Toasts, Animations, Responsive breakpoint: all as documented in v1 - colors, radii, shadows, the amber edit-mode highlight, the 768px breakpoint - unchanged. (Full detail retained below for implementation reference.)

- **Cards**: white background, `border-radius: 8px`, `box-shadow: 0 2px 8px rgba(0,0,0,0.08)`. Stat cards get a 4px navy top border; member cards get a 4px navy left border and lift on hover.
- **Badges**: pill-shaped. Attendance - present `#d4edda`/`#155724`, absent `#f8d7da`/`#721c24`, no-show `#fff3cd`/`#856404`, visitor (new) - a distinct neutral badge, e.g. `#e2e3e5`/`#383d41`, matching the "unknown" proximity treatment for visual consistency. Proximity - local `#d1ecf1`/`#0c5460`, regional `#fff3cd`/`#856404`, abroad `#f8d7da`/`#721c24`, unknown `#e2e3e5`/`#383d41`.
- **Buttons**: `border-radius: 6px`, primary = navy fill/white text, secondary = light-grey fill.
- **Forms**: navy focus ring; read-only fields greyed; the pale-yellow/amber edit-mode highlight (`#fffacd` background, `#ffc107` border) preserved exactly.
- **Modals**: centered overlay, white content card, `border-radius: 12px`. Fullscreen Service Calendar modal preserved.
- **Toasts**: top-right, green success / red error, slide-in, auto-dismiss.
- **Responsive breakpoint**: single breakpoint at `768px`; fully usable phone through desktop.

8.1 Visual Enhancements (accepted - now committed requirements, except dark mode)

Kept deliberately subtle so nothing needs to be relearned - same navy identity, same layout, same interaction patterns. **All of the following are approved and part of the build**, except dark mode: - **Typography refresh**: swap the Segoe UI/Tahoma system-font stack for a modern free web font (e.g., **Inter** via Google Fonts) - same sizes/weights, just crisper

rendering. - **Softer depth**: larger-blur, lower-opacity card shadows and slightly larger corner radii (10-12px instead of 8px) for a more contemporary feel. - **Small line-icons**: next to stat-card labels and section headers (e.g., a people icon for "Total Members," a calendar icon for attendance stats) using a free icon set (Lucide or Heroicons) in the navy palette - not photographic imagery, so it stays fast-loading and visually consistent. - **Skeleton loading states**: replace the plain spinner with content-shaped placeholder blocks while data loads - feels faster and more modern. - **A subtle decorative touch in the header**: a low-opacity geometric or faith-appropriate pattern behind the header content, rather than a flat gradient - adds polish without adding clutter. - **Small data visualizations**: e.g., a simple donut chart for the Local/Regional/Abroad/Unknown proximity breakdown, or a sparkline for the monthly-attendance trend, instead of plain numbers. - **Dark mode - parked, not in this build.** May be added in a future version at the owner's request.

---

9. Non-Functional Requirements

9.1 Real-time updates

Any create/update/delete to shared data should propagate to all other connected users within roughly a second or two via Supabase Realtime subscriptions.

9.2 Zero cost to build and operate

Confirmed achievable at this ministry's scale on the free tiers of Vercel + Supabase - now across **two environments** (production and QA, Section 1.1), both still free at this scale.

9.3 Storage headroom (confirmed)

Current photo library totals ~263 files, ~14-15 MB, against Supabase's 1 GB free file-storage tier (~1.5% used). Even a 10x increase remains comfortably within the free tier.

9.4 Responsive design

Desktop, tablet, and phone must all be fully functional.

9.5 Auth

Google OAuth sign-in (primary) and email/password (supported fallback), both via Supabase Auth.

---

10. Data Migration (fully specified)

10.1 Incremental refresh during the testing period (one-way - Sheets is sole source of truth)

Confirmed: the current Google Sheets app remains the **only** source of truth for real data throughout the testing window - the new app is for navigation/functionality testing only, and anything you create there directly is disposable, not data of record. This is a **one-way sync**, with a rule to handle throwaway test data cleanly, and - as of this revision - a broadened set of tables excluded from the ongoing sweep so that admin configuration work done in the new app isn't overwritten each refresh.

Operational tables - swept on every refresh. These have real, ongoing Sheets activity throughout the testing window (servants keep using the old app), so each refresh makes them an exact mirror of current Sheets content: `members`,

`attendance_records`, `outreach_entries`, `service_calendar_events`, and the contact-info fields of `profiles` (servants).

1. Read the current state of the corresponding sheet/tab via the Sheets API.
2. **Insert** any Sheets row with no matching new-app record yet (matched via `legacy_source_ref`, a stored pointer to that row's origin - sheet name + row identifier).
3. **Update** any new-app record whose Sheets source has changed since last sync, to match the Sheets values exactly.
4. **Delete** any new-app record whose `legacy_source_ref` no longer has a matching Sheets row (the source row was removed).
5. **Delete any new-app record with no `legacy_source_ref` at all** - i.e., a row you created directly in the new app while testing, with no Sheets origin. This handles the "a few test rows I added should be cleared out on refresh" case: since Sheets is the sole source of truth for these tables, anything not traceable back to it is disposable test data, swept away automatically on every refresh.

Configuration tables - excluded from the ongoing sweep. These are admin-curated settings, each with its own maintenance screen in the new app (Section 6.14, Section 6.9, Section 6.1): `universities`, `verses`, `actions_needed_config`, `audit_config` - plus the already-excluded `user_roles`, `app_settings`, `groups`, and `qr_codes` (Section 4.2 covers `user_roles` specifically). All of these are **seeded once from their current Sheets values during the initial migration**, but after that, they're owned entirely by the new app's own admin screens and are never touched by a subsequent refresh - so if you add a university, tweak an Actions Needed threshold, or edit the verse list directly in the new app while testing, a refresh will never undo that. The trade-off, stated plainly: a genuinely new university or verse added on the *Sheets* side after initial migration won't automatically appear in the new app via refresh - it would need to be added directly through the new app's own maintenance screen instead, since these are now admin-owned configuration rather than ongoing synced data. Flag if you'd rather any of these stayed sweep-synced instead.

Role assignments (`user_roles` - who is an Admin/General Coordinator/Sub-Coordinator/Servant, and for which group): the old Permissions sheet doesn't map cleanly onto the new role model (Section 4), so these are set up once, directly in the new app's Access Maintenance screen, and are expected to diverge from (and eventually fully replace) the old sheet.

`groups` and `qr_codes` are new-app concepts with no Sheets equivalent at all, and `audit_log` is the new app's own operational log (not a mirror of anything) - none of these are touched by a refresh; they evolve only through the app's own tools (Group Transition, Admin settings).

10.2 Production Cutover Runbook

Target: the weekend following the last Friday meeting of the Coptic year, **September 11, 2026**.

1. Freeze the Google Apps Script app (no further edits accepted).
2. Notify current app users not to attempt any changes during the cutover window.
3. Run one final data migration - either another incremental refresh, or a full fresh reload, at your discretion at the time.
4. Once that final migration is confirmed complete, trigger the annual Group Transition in the new app to confirm the whole mechanism (Section 5) works correctly end-to-end on real data.
5. Send the new app's link to all users.

6. Users log into the new app and begin normal use; the old Google Sheets app is retired.

---

11. Explicitly Out of Scope / Dropped from the Rebuild

- The orphaned `showServantAssignments()` "Coming soon!" stub - dead code, no UI entry point, unrelated to the working Servant Profiles & Assignments feature which **is** being rebuilt.
- The old two-layer authorization model (app allowlist + separate native spreadsheet sharing) - replaced entirely by Section 4. Cross-group access exceptions, which the old Permissions sheet handled ad hoc, are now handled cleanly by granting an additional role row (Section 4.2, Section 6.14) - no gap here.
- Hardcoded per-year QR code label strings and the 2035-capped Coptic holiday lookup table - replaced by dynamic/algorithmic equivalents.
- The "Access Servants" permissions-sheet column that existed but was never actually read by any function.

---

12. Open Assumptions (flagged for review)

Resolved (kept here as a record, not open anymore): visitors excluded from Dashboard stats/Actions Needed - confirmed correct. Collapsible-section state persisting per session only - confirmed correct. Visual enhancements (Section 8.1) - all accepted except dark mode, which is parked. Data migration is one-way, Sheets-is-sole-source-of-truth, with new-app-only test rows swept on every refresh (Section 10.1) - confirmed correct. Year-0 QR labeling (Section 2.2, Section 6.11) - confirmed, was a typo.

Still open: - Gender-based filtering in servant-assignment dropdowns remains a soft suggestion, not a hard block. - Both Categorical and Alphabetical servant-directory view modes are retained. - Audit log "archiving by age" may remain a hard-delete-after-N-months operation rather than moving records to cold storage. - New-member and attendance intake forms get sensible field-level validation without every rule being individually specified here.

---

13. Working Agreement for This Project (versioning workflow added)

Whenever the app owner reports a bug or questions a decision in the *rebuilt* app: respond with a plan to fix/change it, get explicit approval, implement the change, and then **update this requirements document** so it stays the single, current source of truth for how the app is supposed to behave.

App version numbering (new): the app's own displayed version number (header corner, Section 2.1) is independent of Vercel's deployment history - it starts at **4.0**. Every time a change ships, you'll be prompted to decide the new version number: **+0.1 for a small change or fix** (e.g., 4.0 -> 4.1), or **the next whole integer for a major release** (e.g., 4.x -> 5.0). The chosen version is written to `app_settings.app_version` and reflected in the header, and this document is updated to note the change alongside its version number.